

SMART CONTRACTS: SOLIDITY SYNTAX

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

Isaac Oh

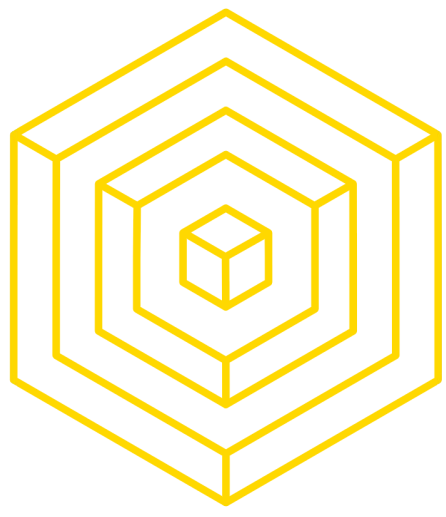
Aman Shah

Elson Liu

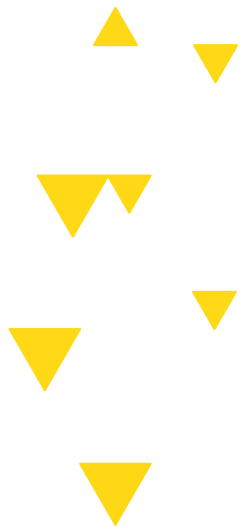
Fujia Wang



BLOCKCHAIN
AT BERKELEY



O Announcements





Announcements

- Homework #1
 - Due today at 11:59PM
- Homework #2
 - Due Friday at 11:59PM
 - Since instructions were using Goerli instead of Sepolia
- Homework #3 Released!
 - Due next Wednesday at 11:59PM
 - Implementing an NFT Marketplace
- All submission links are posted on the Discord and Github
- Email dev-decal@blockchain.berkeley.edu for extensions



LECTURE OVERVIEW

- 1 SOLIDITY OVERVIEW
- 2 GLOBAL VARIABLES
- 3 EXTERNAL CONTRACTS
- 4 MODIFIERS
- 5 EVENTS, STORAGE, & UPGRADABILITY



1 SOLIDITY OVERVIEW



THE BANK CONTRACT

LEARNING FAST

WHAT DOES A BANK NEED TO DO?

1. Allow Deposits
2. Allow Withdrawals
3. Balance Checks

[Learn X in Y minutes](#), a whirlwind
tour of your favorite language

```
1 contract SimpleBank {
2     mapping (address => uint) private balances;
3     address public owner;
4     event LogDepositMade(address accountAddress, uint amount);
5
6     function SimpleBank() {
7         owner = msg.sender;
8     }
9
10    function deposit() public returns (uint) {
11        balances[msg.sender] += msg.value;
12        LogDepositMade(msg.sender, msg.value);
13        return balances[msg.sender];
14    }
15
16    function withdraw(uint withdrawAmount) public returns (uint remainingBal) {
17        if(balances[msg.sender] >= withdrawAmount) {
18            balances[msg.sender] -= withdrawAmount;
19            if (!msg.sender.send(withdrawAmount)) {
20                balances[msg.sender] += withdrawAmount;
21            }
22        }
23        return balances[msg.sender];
24    }
25
26    function balance() constant returns (uint) {
27        return balances[msg.sender];
28    }
29
30    function () {
31        throw;
32    }
33 }
```




THE BANK CONTRACT

LEARNING FAST

1
0

- **contract** has similarities to **class** in other languages (class variables, inheritance, etc.)
 - Declare state variables outside function, persist through life of contract
- **mapping** is a dictionary that maps addresses to balances
 - always be careful about overflow attacks with numbers
 - private means that other contracts can't directly query balances
 - but data is still viewable to other parties on blockchain
- **public** makes externally readable (not writeable) by users or contracts

```
1 contract SimpleBank {  
2  
3  
4  
5     mapping (address => uint) private balances;  
6  
7  
8  
9  
10    address public owner;  
11  
12
```



THE BANK CONTRACT

LEARNING FAST

Visibility & Access Modifiers

Keyword	Meaning / Purpose	Notes / Clarifications
<code>public</code>	Function/variable can be accessed internally <i>and externally</i> .	Generates an automatic <i>getter</i> for state variables.
<code>private</code>	Accessible only inside the contract where it's defined.	Still visible on-chain — “private” means contract-level, not blockchain-level privacy.
<code>internal</code>	Accessible only inside the contract and contracts that inherit from it.	Similar to <code>protected</code> in OOP.
<code>external</code>	Functions accessible only from <i>outside</i> the contract.	Can't be called internally unless using <code>this.myFunction()</code> .



THE BANK CONTRACT

LEARNING FAST

- **event** - publicize actions to external listeners
- **Constructor** - can receive one or many variables here; only one allowed
- **msg** provides details about the message that's sent to the contract
 - **msg.sender** is contract caller (address of contract creator)

```
4     event LogDepositMade(address accountAddress, uint amount);  
5  
6     function SimpleBank() {  
7         owner = msg.sender;  
8     }
```



THE BANK CONTRACT

LEARNING FAST

1
2

- **deposit()**
 - Takes no parameters, but we are still sending Ether!
 - public makes externally readable (not writeable) by users or contracts
 - Returns user's balance as an unsigned integer (uint)
- balances[msg.sender], no this or self required with state variable
- LogDepositMade event fired

```
14- function deposit() public returns (uint) {  
15-  
16-  
17-     balances[msg.sender] += msg.value;  
18-  
19-  
20-     LogDepositMade(msg.sender, msg.value);  
21-  
22-  
23-     return balances[msg.sender];  
24- }  
25-
```




THE BANK CONTRACT

LEARNING FAST

https://docs.google.com/presentation/d/1otq17RqVWhZYzhsqy9qhp9xkwINnX01d1v71IMX7Y0HI/edit#slide=id.g3B62d1187_1_85

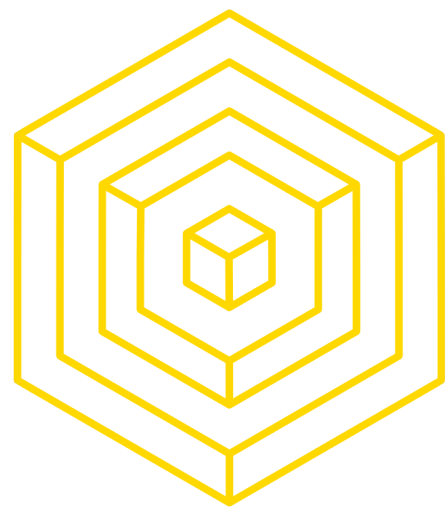
1
3

- **withdraw()**
 - **withdrawAmount** parameter
 - Returns user's balance
- Note the way we deduct the balance right away, before sending
 - We do this because of the risk of a recursive call that allows the caller to request an amount greater than their balance
- Increment back only on fail, as may be sending to contract that has overridden 'send' on the receipt end

```
function withdraw(uint withdrawAmount) public returns (uint remainingBalance) {  
  
    if(balances[msg.sender] >= withdrawAmount) {  
  
        balances[msg.sender] -= withdrawAmount;  
  
        if (!msg.sender.send(withdrawAmount)) {  
  
            balances[msg.sender] += withdrawAmount;  
        }  
    }  
    return balances[msg.sender];  
}
```



2 GLOBAL VARIABLES

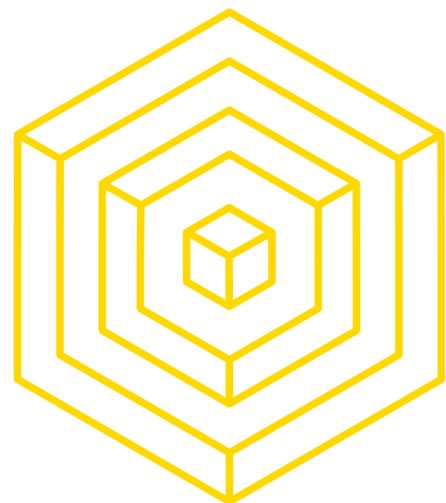


GLOBAL VARIABLES

CURRENCY UNITS



- In Solidity, currency units are tracked as **uints**
- A number can take a postfix of **wei**, **finney**, **szabo** or **ether** to convert between denominations of Ether.



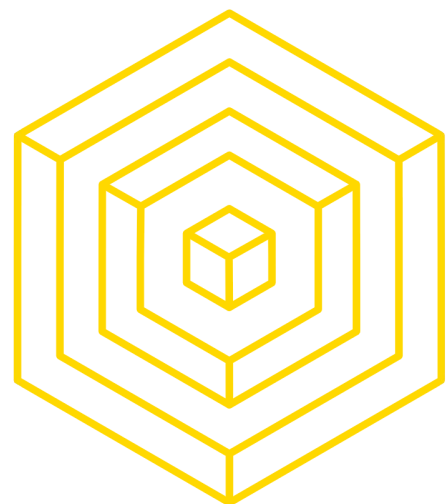
GLOBAL VARIABLES

CURRENCY UNITS

```
// Currency units
// Currency is defined using wei, smallest unit of Ether
uint minAmount = 1 wei;
uint a = 1 ether; // 1 ether == 10**18 wei
uint b = 1 finney; // 1 ether == 1000 finney

// Currency units without a postfix are assumed to be wei
require(a == 10**15 && b == 10**18); // true
```

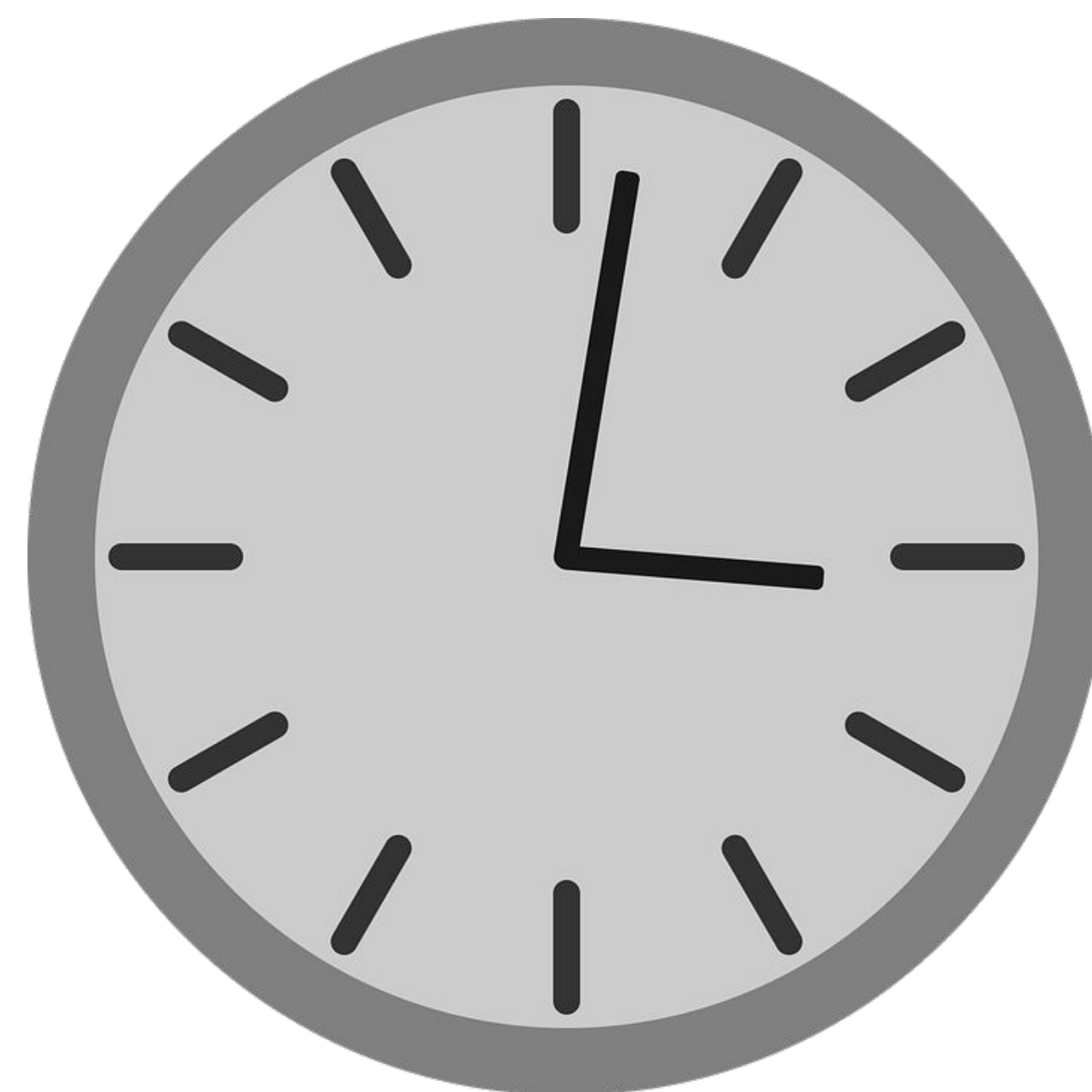


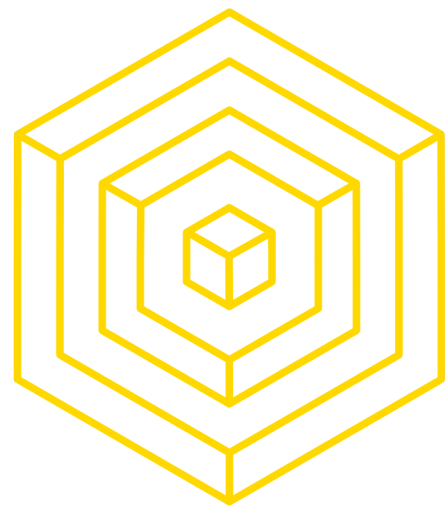


GLOBAL VARIABLES

TIME

- Solidity tracks time as a **Unix TimeStamp**, stored as a **uint**
 - Find current timestamp:
<https://www.unixtimestamp.com/index.php>
- Similar to currency units, a number can take a postfix of **seconds, minutes, hours, etc.** to convert between time denominations





GLOBAL VARIABLES

TIME

```
// Time units
```

```
1 == 1 seconds
```

```
1 minutes == 60 seconds
```

```
1 hours == 60 minutes
```

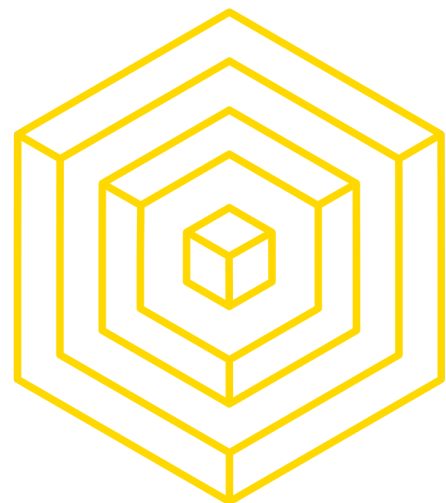
```
1 days == 24 hours
```

```
1 weeks == 7 days
```

```
1 years == 365 days
```

```
now; // returns current Unix TimeStamp
```

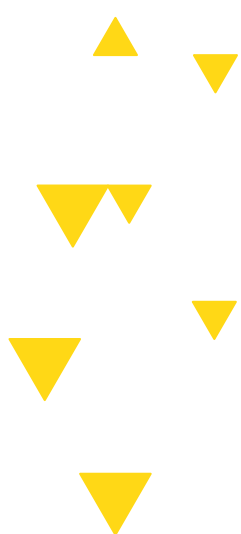
```
// Note that this can be manipulated by miners, so use carefully
```

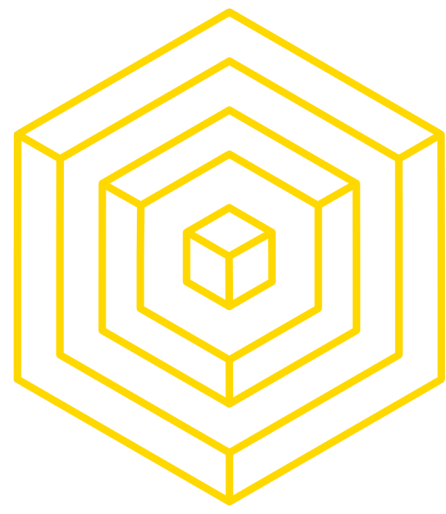


GLOBAL VARIABLES

THIS

```
// ** this **  
this; // address of contract  
// often used at end of contract life to transfer remaining balance to party  
this.balance;  
this.someFunction(); // calls func externally via call, not via internal jump
```



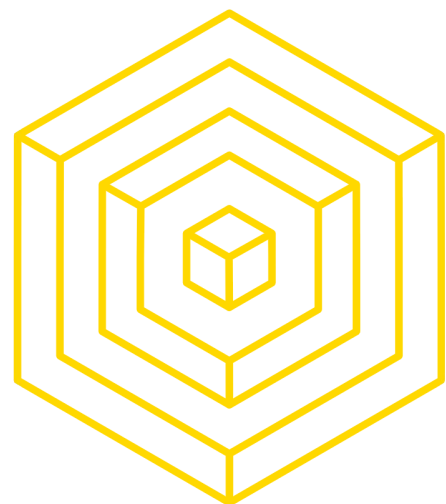


GLOBAL VARIABLES

MSG, TX

```
// ** msg - Current message received by the contract ** **  
msg.sender; // address of sender  
msg.value; // amount of ether provided to this contract in wei, function should be marked "payable"  
msg.data; // bytes, complete call data  
msg.gas; // remaining gas
```

```
// ** tx - This transaction **  
tx.origin; // address of sender of the transaction  
tx.gasprice; // gas price of the transaction
```



GLOBAL VARIABLES

MSG.SENDER VS TX.ORIGIN



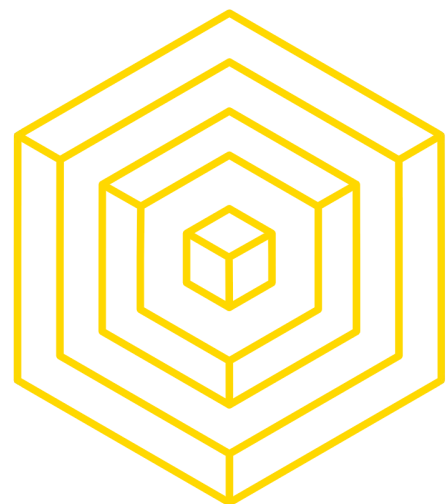
**Externally owned
account at address A**



Contract at address B



Contract at address C



GLOBAL VARIABLES

MSG.SENDER VS TX.ORIGIN



**Externally owned
account at address A**

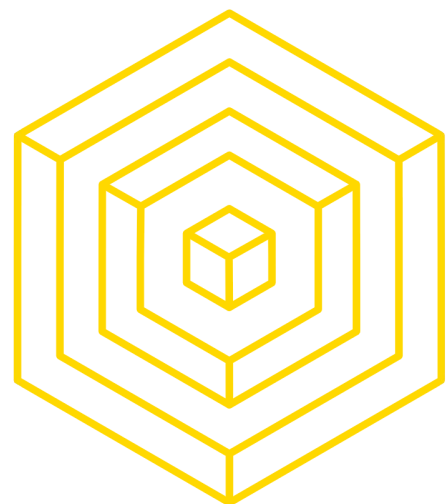


Contract at address B



Contract at address C

Msg.sender: address A
Tx.origin: address A



GLOBAL VARIABLES

MSG.SENDER VS TX.ORIGIN



**Externally owned
account at address A**



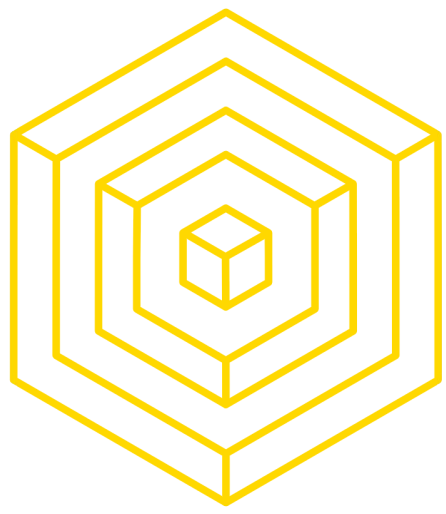
Contract at address B

Msg.sender: address A
Tx.origin: address A

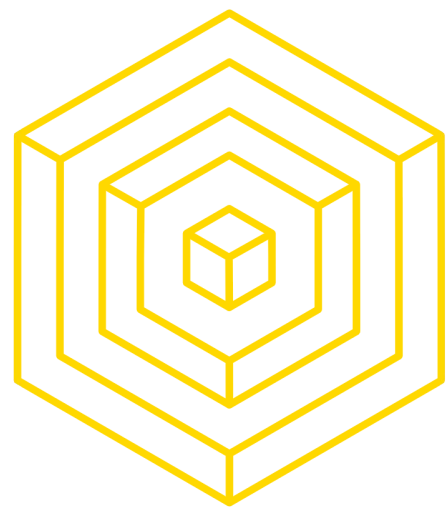


Contract at address C

Msg.sender: address B
Tx.origin: address A



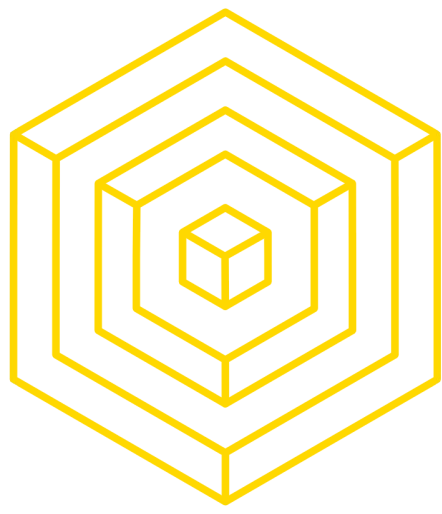
3 EXTERNAL CONTRACTS



EXTERNAL CONTRACTS

EXTERNAL CONTRACTS

```
contract InfoFeed {  
    function info() returns (uint) {  
        return 42;  
    }  
}
```

EXTERNAL CONTRACTS

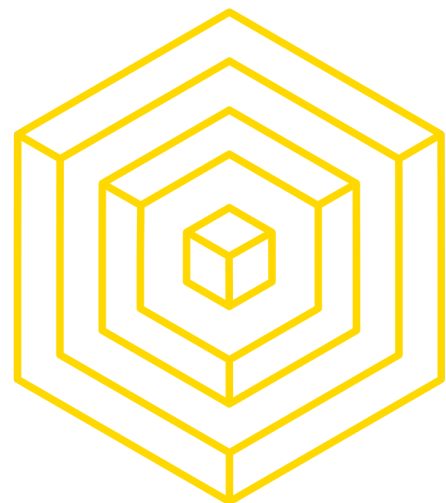
EXTERNAL CONTRACTS

```
contract InfoFeed {  
    function info() returns (uint) {  
        return 42;  
    }  
}
```

```
// Import the file if contracts are in separate files  
import "../InfoFeed.sol";  
  
contract Consumer {  
    InfoFeed feed; // points to contract on blockchain  
  
    // Set feed to new instance of contract  
    function createNewFeed() {  
        // new instance created, constructor call  
        feed = new InfoFeed();  
    }  
  
    // Set feed to existing contract instance  
    function setFeed(address addr) {  
        feed = InfoFeed(addr);  
    }  
  
    function callFeed() {  
        feed.info();  
    }  
}
```



4 MODIFIERS

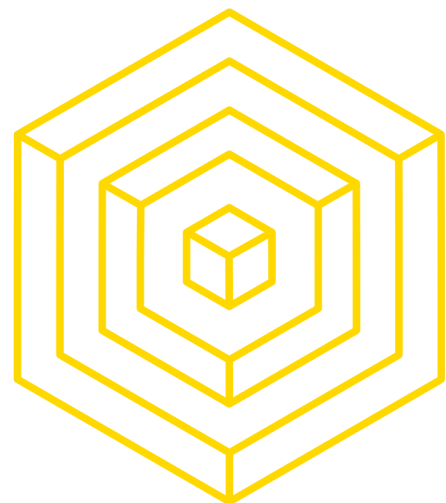


MODIFIERS

MODIFIERS

```
// Modifiers validate inputs to functions such as minimal balance or user auth;
```

```
// Transfers balances from one address to another  
function transferBalance(address from, address to) public onlyOwner {  
    balances[to] += balances[from];  
    balances[from] = 0;  
}
```

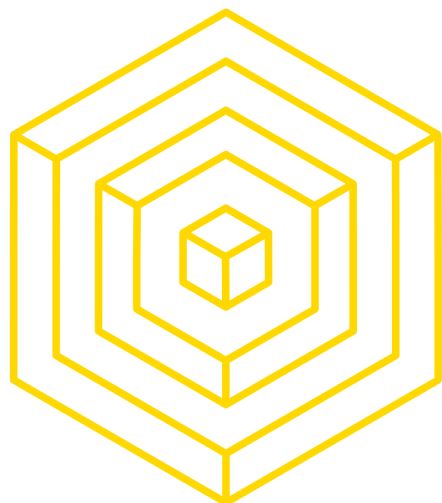
MODIFIERS

MODIFIERS

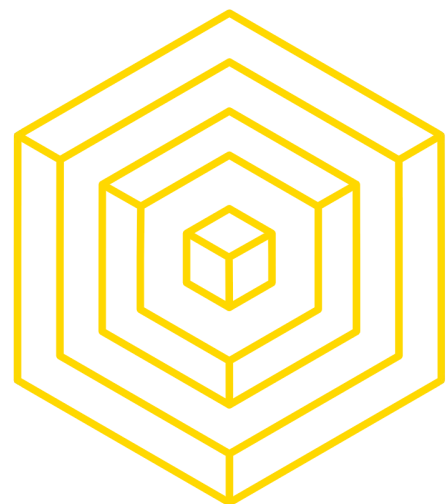
```
// '_' (underscore) often included as last line in body, and indicates  
// the function being called should be placed there
```

```
modifier onlyOwner() {  
    require(msg.sender == owner);  
    _;  
}
```

```
modifier onlyAfter(uint _time) {  
    require (now >= _time);  
    _;  
}
```



5 Events



Events

What are events?

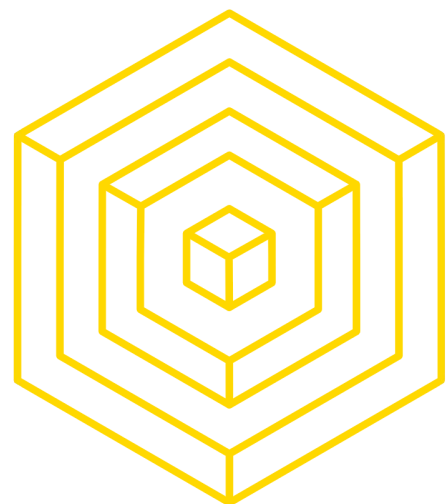
- Logging mechanism to EVM
- Notifies others about important occurrences

Event declaration

```
event Deposit(  
    address indexed user,  
    uint256 etherAmount,  
    uint256 time  
);
```

Emitting an event

```
function deposit() public payable {  
    /*  
    Some other code...  
    */  
  
    emit Deposit(msg.sender, msg.value, block.timestamp);  
}
```

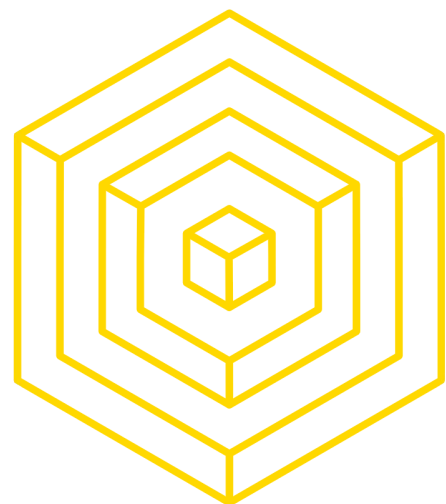
Events

Why use events?

- Efficiently read blockchain activity
- Interface between smart contracts and dApps
- External system compatibility
- Ex. MEV bot listening for event on Uniswap liquidity pool

Uniswap event

```
49 event Mint(address indexed sender, uint amount0, uint amount1);
50 event Burn(address indexed sender, uint amount0, uint amount1, address indexed to);
51 event Swap(
52     address indexed sender,
53     uint amount0In,
54     uint amount1In,
55     uint amount0Out,
56     uint amount1Out,
57     address indexed to
58 );
59 event Sync(uint112 reserve0, uint112 reserve1);
```



Events

Looking at a log

Event declaration

```
event LogPurchase(  
    address indexed buyer,  
    uint256 amount  
);
```

Event emission

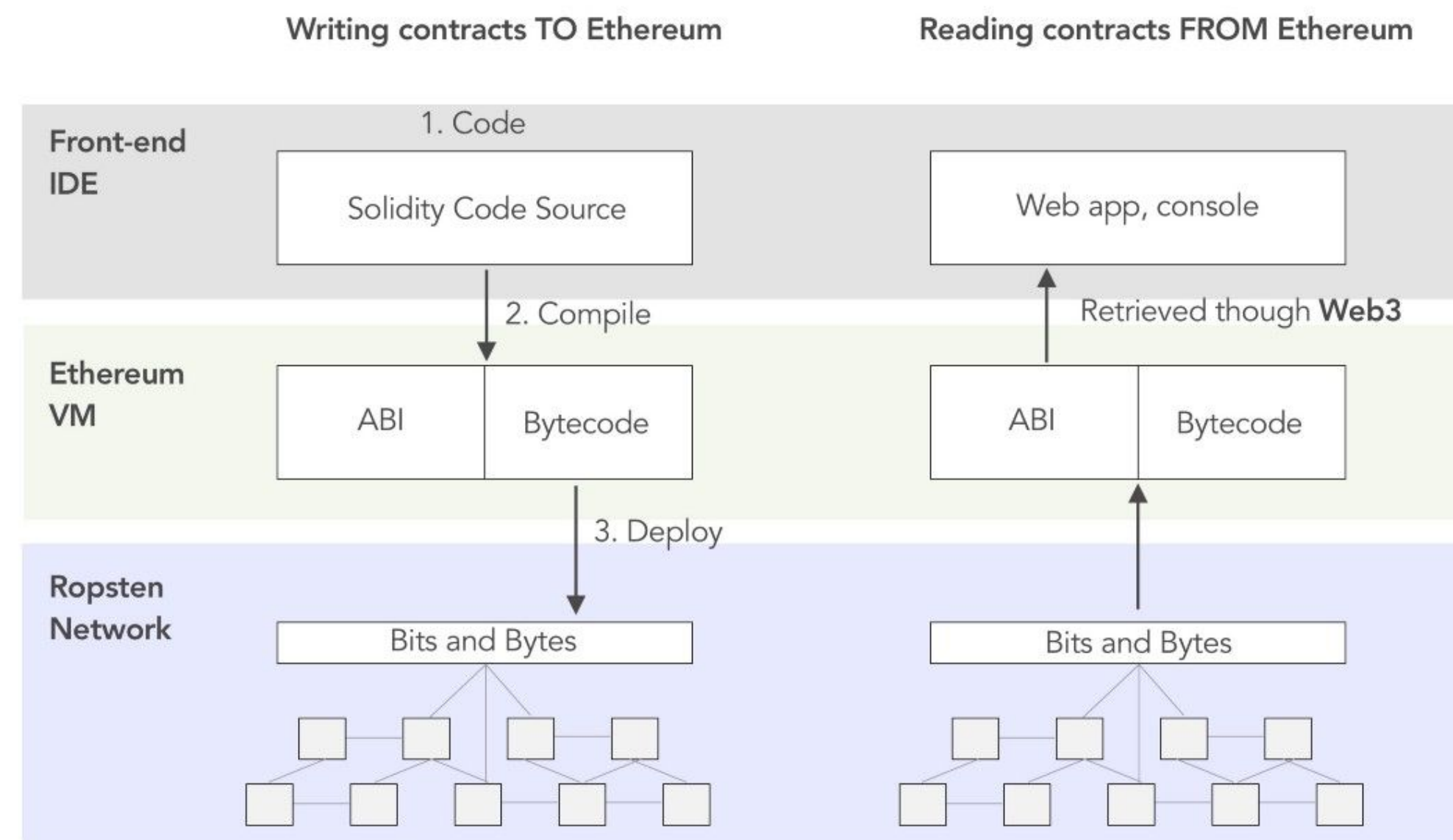
```
function buy() public payable {  
    // ... some logic ...  
    emit LogPurchase(msg.sender, msg.value);  
}
```

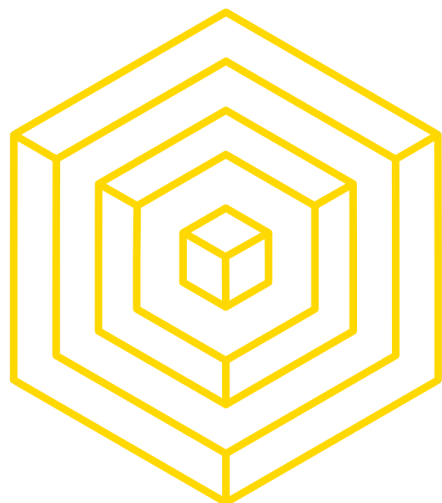
Logged output

```
{  
  "logIndex": "0x1", # Index position of the log  
  "transactionIndex": "0x10", # Index position of the transaction in the block  
  "transactionHash": "0xa123b456...", # Hash of the transaction  
  "blockHash": "0xdeadbeef...", # Hash of the block where this log was in  
  "blockNumber": "0x5b5b", # Block number where this log was in  
  "address": "0xcontractAddress...", # Address of the contract that produced the log  
  "data": "0x0000000000000000000000000000000000000000000000000000000000000005", # Data associated with the log  
  "topics": [  
    "0xkeccak256HashOfLogPurchaseSignature", # This is a hash of the event signature  
    "0xAbc123..." # Indexed parameter  
  ],  
  "type": "mined"  
}
```


Working with a log

- Decode `data` and `topics` field using ABI (Application Binary Interface) of the contract
 - ABI converts bytecode

[illegible]



6 Storage



Types of Memory

- Memory
 - Used for variables and parameters within functions.
 - Exists only during the lifetime of a function.
 - Variables disappear after function execution.
 - Commonly understood memory type for those with programming backgrounds.
- Calldata
 - Used for external function's
 - Similar to memory but is read-only.
 - References a specific area of memory.
- Storage
 - Contract's long-term memory.
 - Retains variables after function or transaction execution.
 - Used for contract's state variables (outside functions).
 - Unique to blockchains: tamper-proof due to cryptographic sealing.
 - In other environments, similar data is stored in filesystems or databases.
 - On blockchain, code and data are persisted together long-term.

Function visibility	Data location for function parameter can be
external	storage = ✗ not allowed memory = ✓ (since 0.6.9) calldata = ✓
public	storage = ✗ not allowed memory = ✓ calldata = ✓ (since 0.6.9)
internal	storage = ✓ memory = ✓ calldata = ✓ (since 0.6.9)
private	storage = ✓ memory = ✓ calldata = ✓ (since 0.6.9)

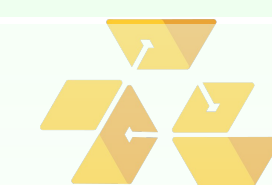


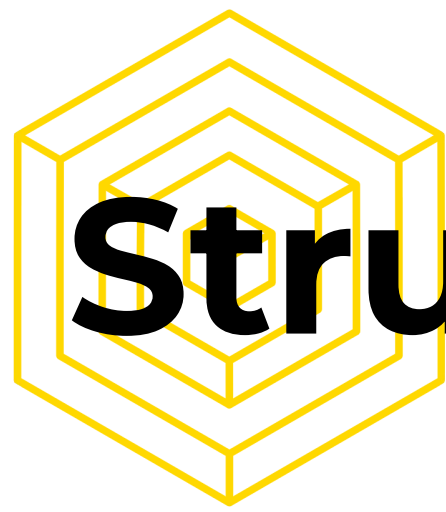
Types of Memory

- Memory
 - Used for variables
 - Exists only while program is running
 - Variables declared in the program
 - Commonly used for temporary storage
- Calldata
 - Used for external data
 - Similar to memory
 - References to data
- Storage
 - Contract's permanent data
 - Retains value even after program ends
 - Used for contract state
 - Unique to each contract
 - Sealing.
 - In other environments
 - databases.
 - On blockchain



er can be

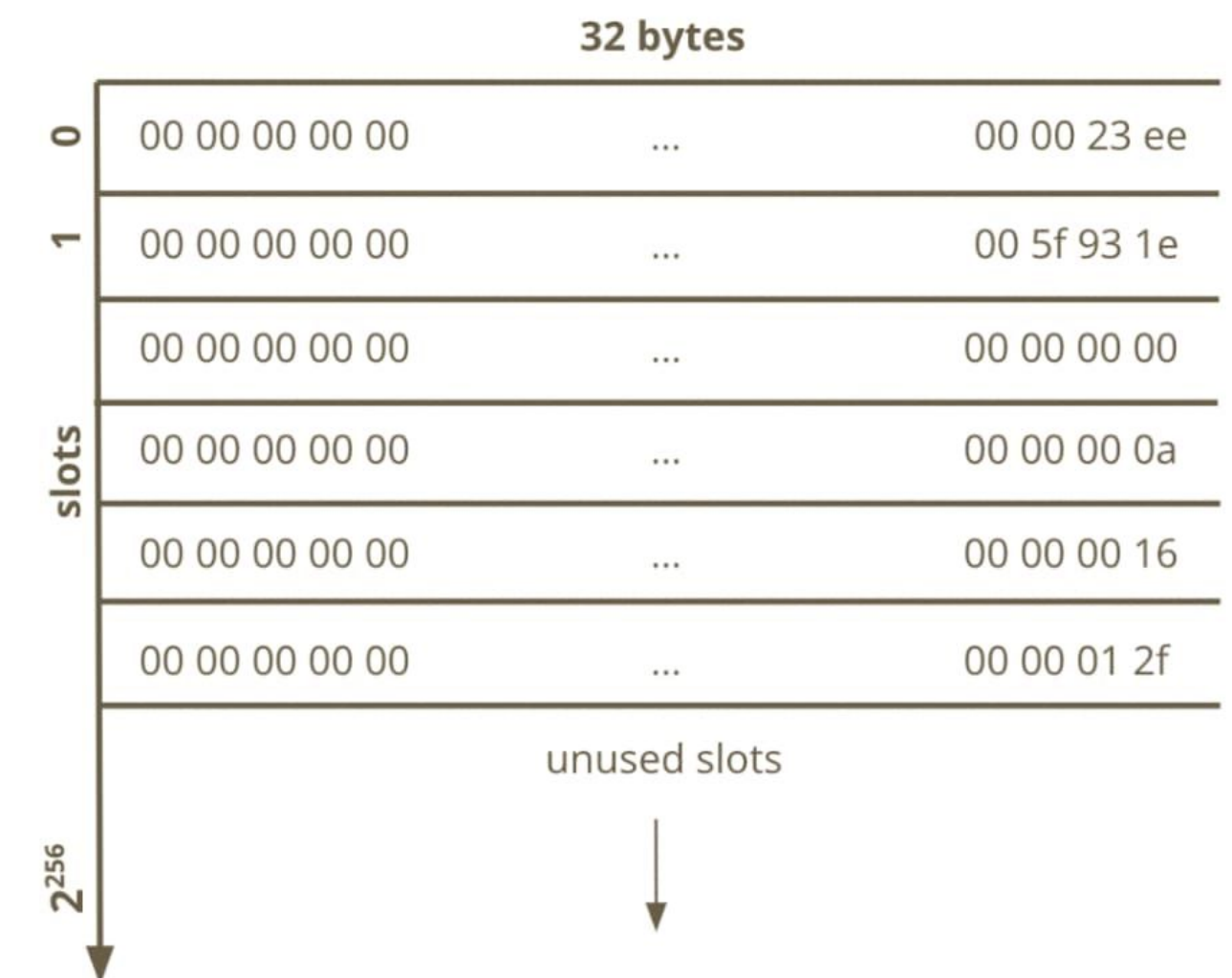


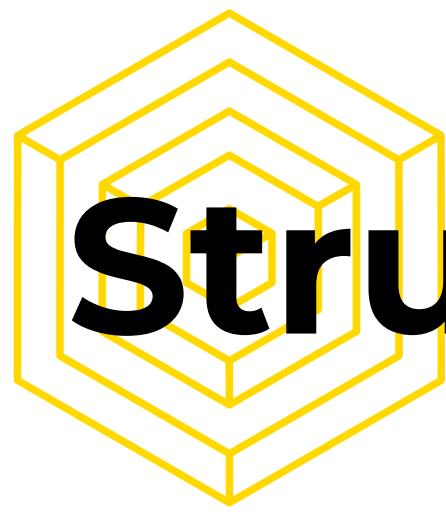


Structured Storage - Storage Keyword

- Ethereum Virtual Machine (EVM)
 - Each smart contract has its own storage.
 - Storage is a key-value store.
 - Each key and value is 32 bytes.
- Storage Layout
 - Determined by order and type of state variables.
 - Each state variable gets a specific storage slot.
 - Slot assignment based on order of declaration.
- 2^{256} slots!

```
contract ValueTypeContract {
    uint256 a = 9198;      // 0x23ee in hexadecimal
    uint256 b = 0x5f931e; // 6263582 in decimal
    bytes32 c;
    uint256[3] d = [ 10, 22, 303 ];
}
```





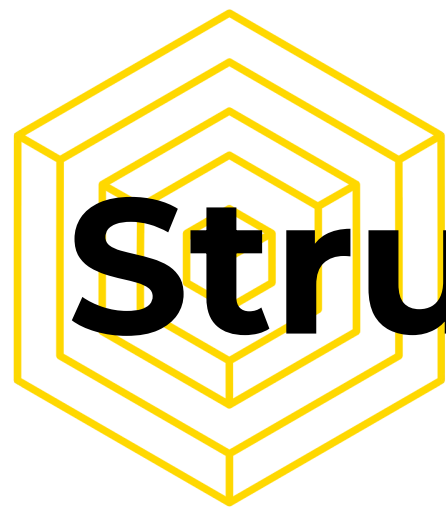
Structured Storage - Storage Keyword

- Ethereum Virtual Machine
 - Each smart contract has a storage layer
 - Storage is a key-value store
 - Each key is 32 bytes
- Storage Layout
 - Determine the layout of variables.
 - Each state variable is stored in a slot.
 - Slot assignment is done at compilation time

TAMPER

PROOF

s	
	00 00 23 ee
	00 5f 93 1e
	00 00 00 00
	00 00 00 0a
	00 00 00 16
	00 00 01 2f
ots	

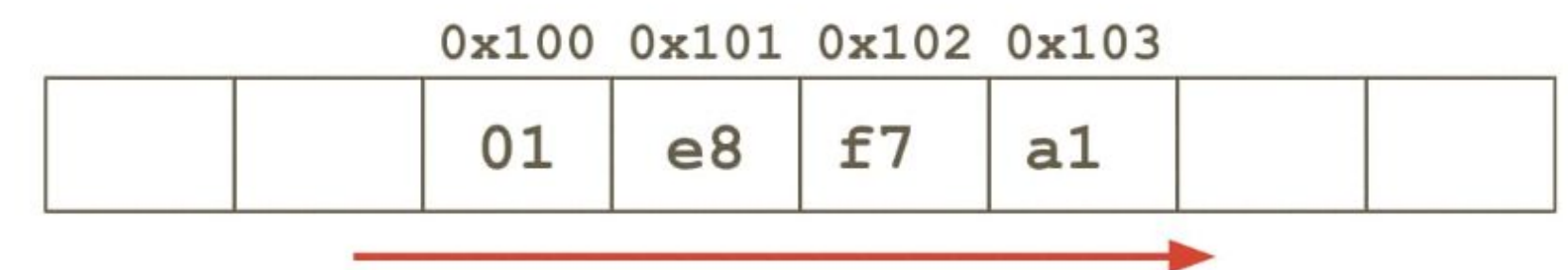


Structured Storage Continued

61C Concepts

- Big-endian is only used for bytes and string types. These 2 types behave differently in a contract's storage slots than other variables.
- Little-endian is used for every other type of variable. Some examples are: uint8, uint32, uint256, int8, boolean, address, etc...

Big-endian



Little-endian

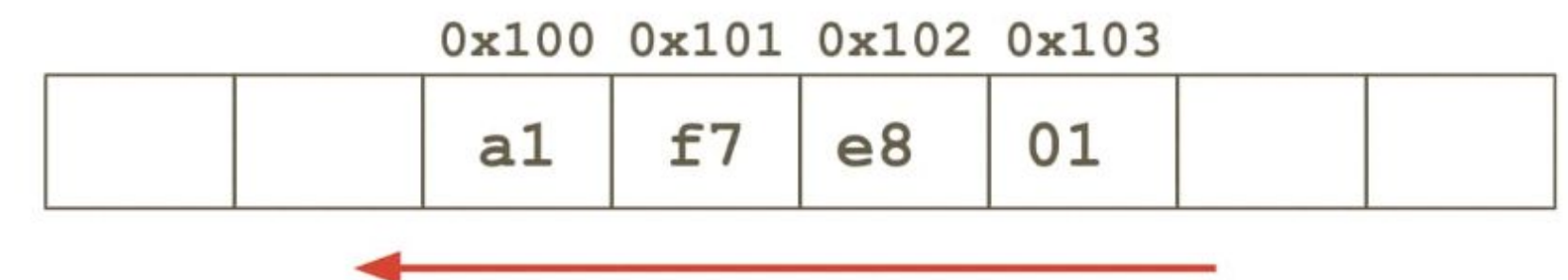


Diagram of how computers store multi-byte values in memory.



Unstructured Storage

- Ethereum Virtual Machine (EVM)
 - Each smart contract has its own storage.
 - Storage is a key-value store.
 - Each key and value is 32 bytes.
- Storage Layout
 - Determined by order and type of state variables.
 - Each state variable gets a specific storage slot.
 - Slot assignment based on order of declaration.

```
contract ValueTypeContract {
    uint256 a = 9198;      // 0x23ee in hexadecimal
    uint256 b = 0x5f931e;  // 6263582 in decimal
    bytes32 c;
    uint256[3] d = [ 10, 22, 303 ];
}
```



Why it's Important

- Be conscious of the ordering of your variables
- As seen on the right you can save storage by ordering correctly, this in turn saves a lot of gas

```
contract PaddedContract {
    bool a = true;        // 1 bit of data
    uint256 b = 0x9b0e11; // 32 bytes of data
    uint8 c = 0xff;       // 1 byte of data
}
```



Diagram of how variables that require less than 32 bytes of memory are stored.

```
contract PackedContract {
    bool a = true;        // 1 bit of data
    uint8 c = 0xff;       // 1 byte of data
    uint256 b = 0x9b0e11; // 32 bytes of data
}
```



Diagram of how the Ethereum Virtual Machine packs variables into storage slots to reduce how much storage memory is used.



Walkthrough Example

```
contract StructContract {
    struct SomeStruct {
        uint256 firstValue;
        uint16 secondValue;
        bool thirdValue;
    }

    uint256 a = 9198;      // 0x23ee in hexadecimal
    uint256 b = 0x5f931e;  // 6263582 in decimal
    SomeStruct structVariable = SomeStruct(
        0x452, 5, true
    );
    bytes32 c;
}
```

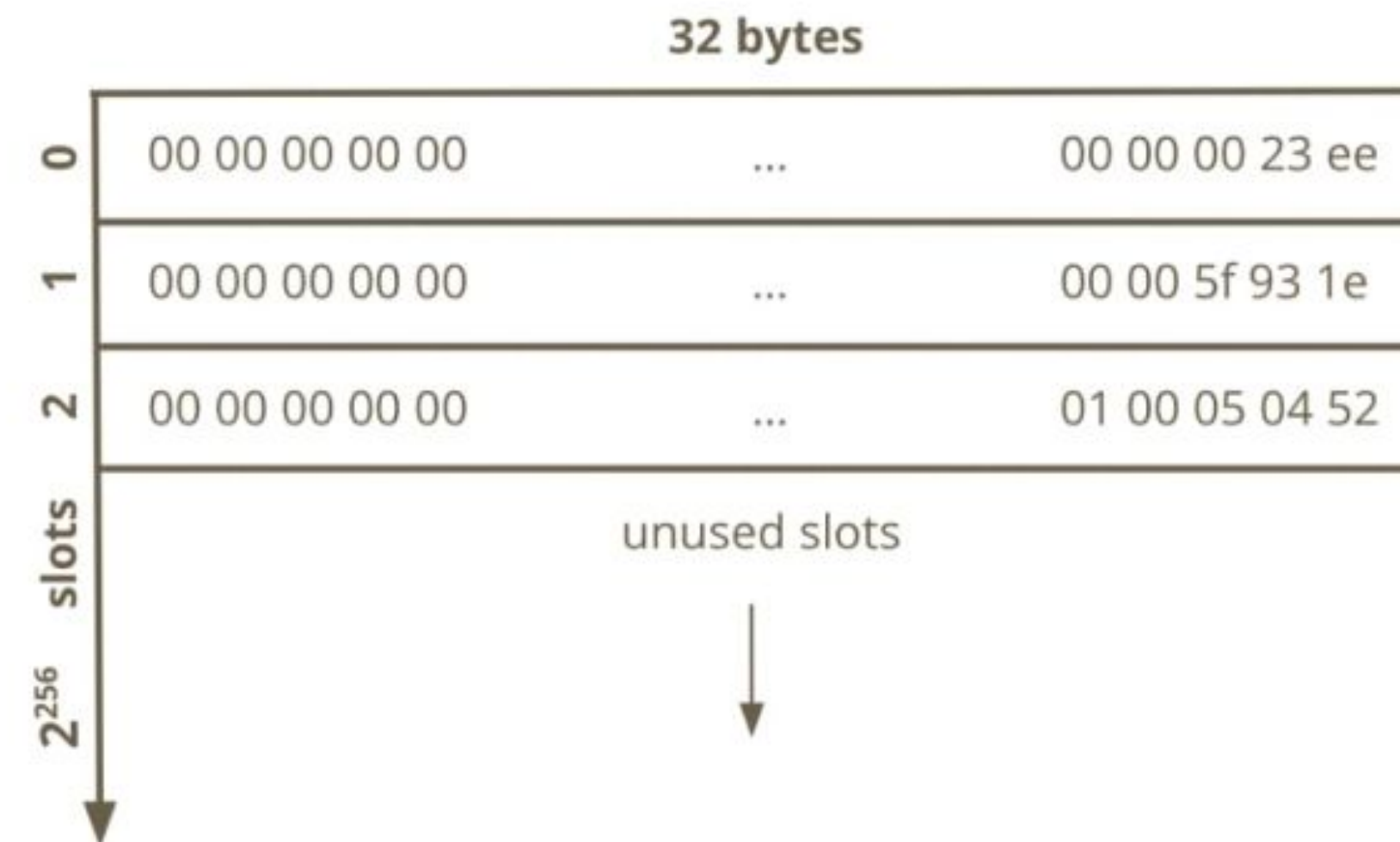


Diagram of how structs are packed and stored in smart contract storage memory.



Statically Stored Variables

```
contract ValueTypeContract {
  uint256 a = 9198;      // 0x23ee in hexadecimal
  uint256 b = 0x5f931e;  // 6263582 in decimal
  bytes32 c;
  uint256[3] d = [ 10, 22, 303 ];
}
```



Diagram of how statically-sized state variables are stored in storage slots.



Dynamically Stored Variables

- Use Keccak256 and Ethereum's “infinite” memory
- The slots for dynamic variables are only marker slots

```
contract DynamicArrayContract {
  uint256 a = 0x23ee; // 0x0
  uint256 b = 0x5f931e; // 0x1
  uint256[] c; // 0x2
  uint256 d = 0x7a01; // 0x3

  constructor() public {
    c.push(0xa);
    c.push(0xf1092);
    c.push(0xb057);
  }
}
```

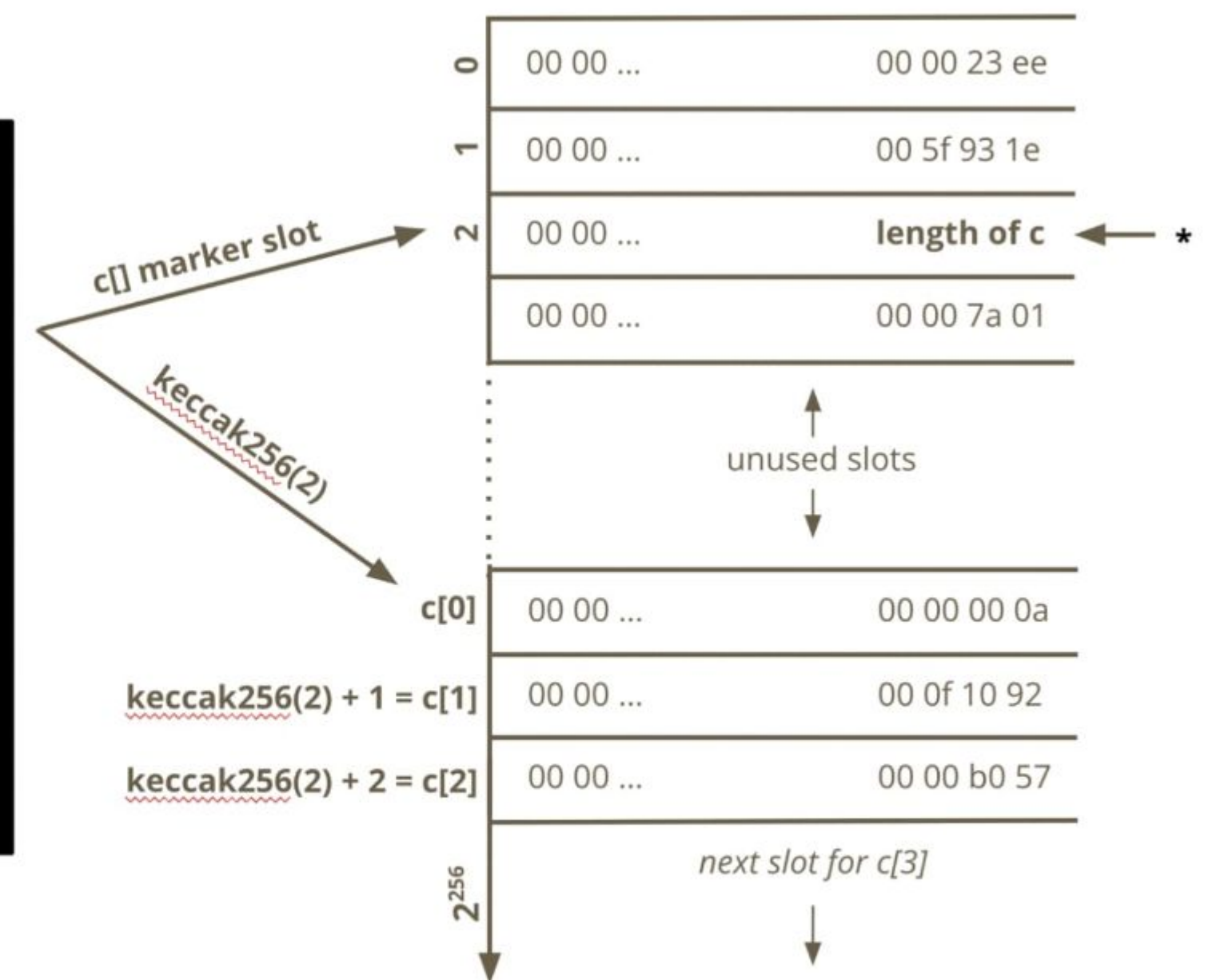


Diagram of how dynamically-sized state variables are stored in storage slots using marker slots and keccak256 hashes.

Mappings

For [mappings](#), the marker slot only marks the fact that there is a mapping. To find a value for a given key, the formula $\text{keccak256}(h(k) \cdot p)$ is used where:

- the symbol $\cdot$ denotes string concatenation
- p is the state variable's declaration position in the smart contract
- $h()$ is a function that is applied to the key depending on the key's type
- for value types, $h()$ returns the padded the value to 32 bytes
- for strings and byte arrays, $h()$ just returns the unpadded data

```
contract MappingsContract {
    uint256 a = 0x23ee;    // 0x0
    uint256 b = 0x5f931e;  // 0x1
    mapping (uint => uint) c; // 0x2
    uint256 d = 0x7a01;    // 0x3

    constructor() public {
        c[30] = 0xd3ff;

        c[122] = 0x101;
    }
}
```

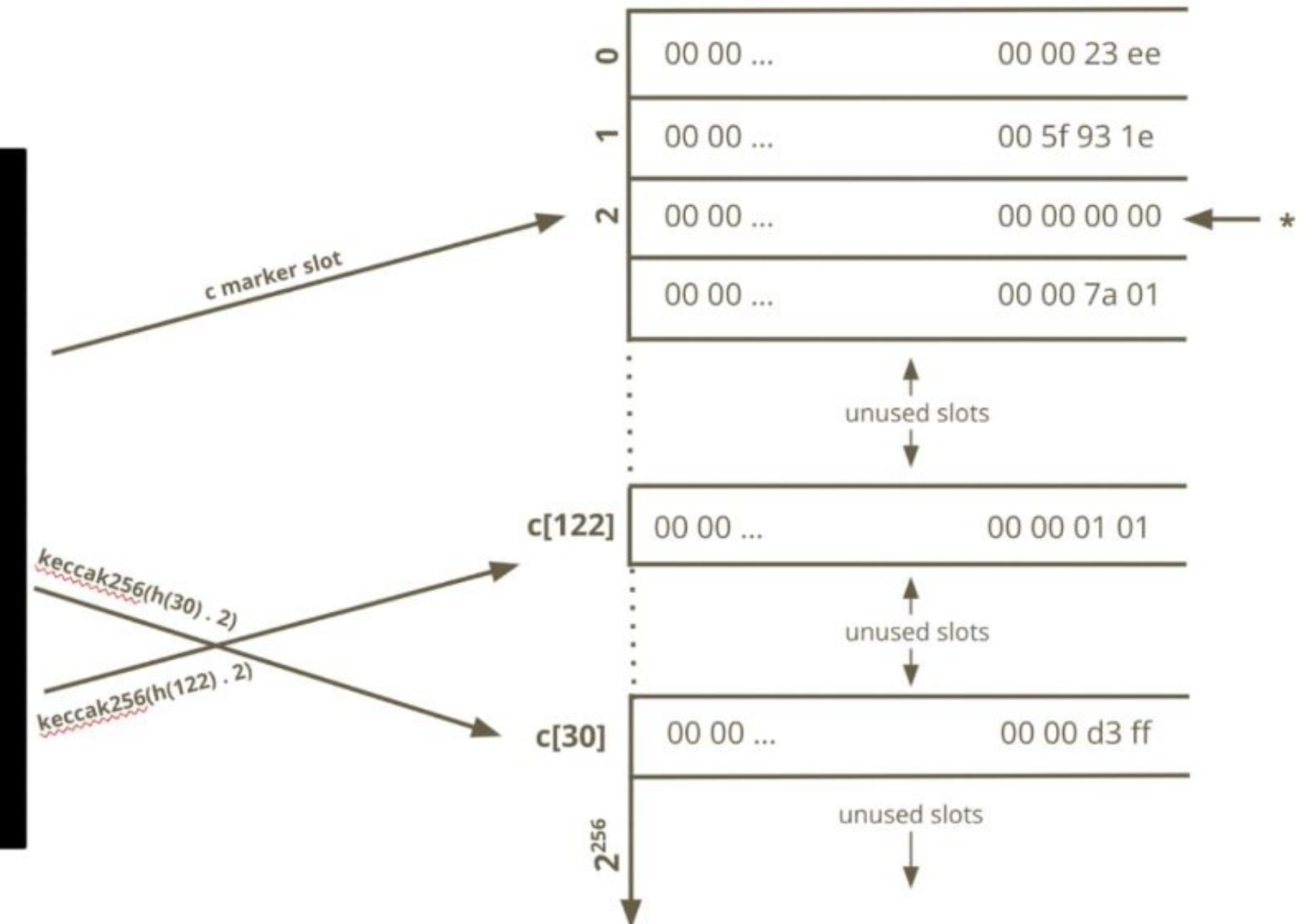
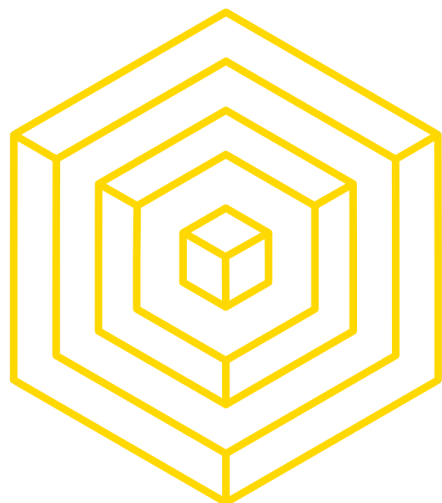
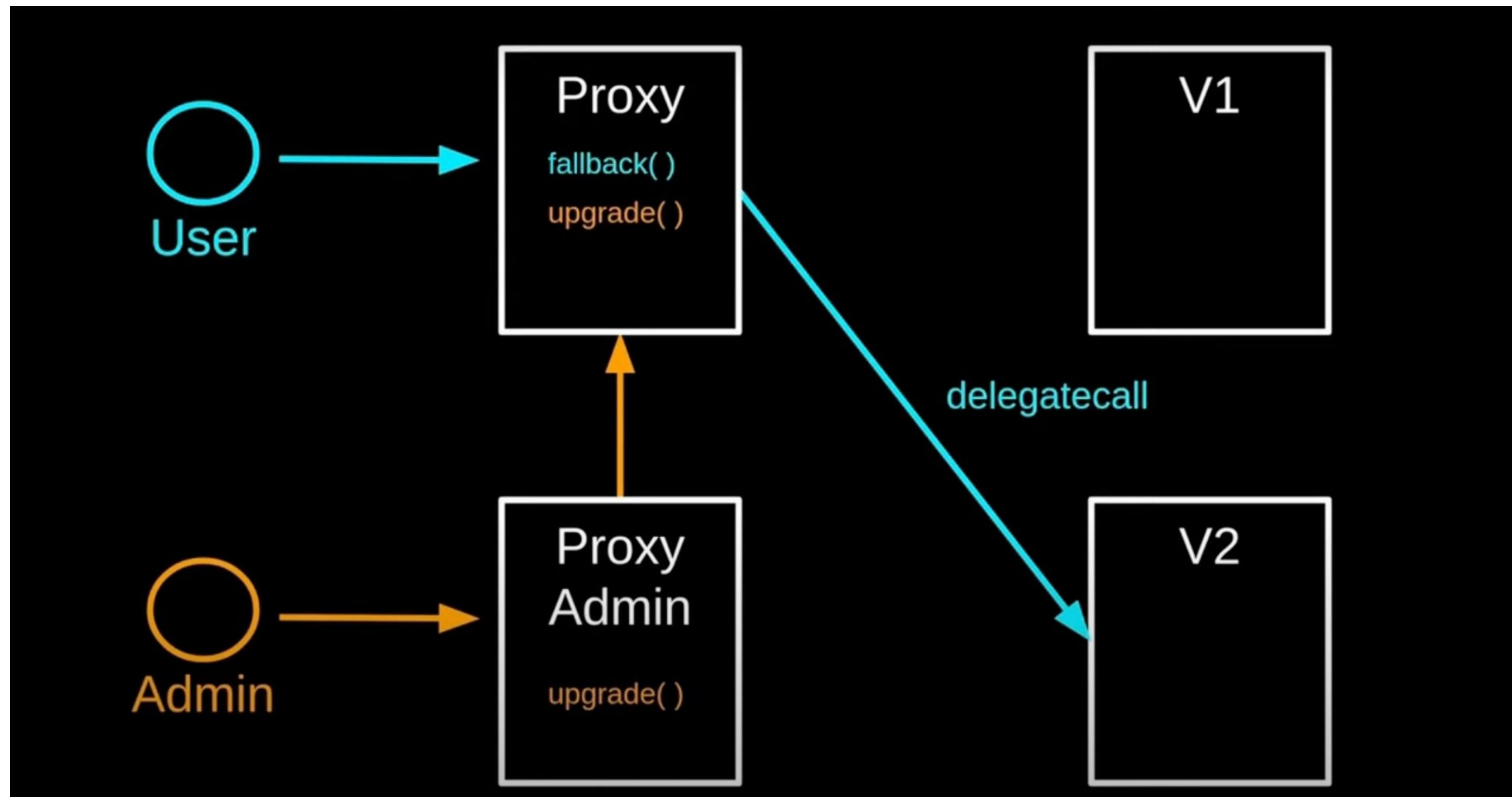


Diagram of how mappings are stored in smart contract storage slots.



7 Upgradeability

Upgradeability





Vitamin/Attendance





Upgradeability

Finish Next Class...